# NestJS

## Zero to Hero

A complete instructor-style guide

Theory • Code • Graphviz diagrams

*Build TaskFlow, a real task-manager API, from an empty folder to JWT auth, PostgreSQL, tests and production polish.*

**What is inside**

- 16 episodes taking you from zero to production patterns
- Running project: TaskFlow API (REST, TypeORM, PostgreSQL, JWT)
- 12 diagrams generated with Graphviz
- Cheat sheets, common-error table, glossary and a 4-week roadmap

October 2026 • Code targets current NestJS releases; check docs.nestjs.com for version changes

# Contents


**Episode 0: Welcome and the Plan . . . 5**

What we are building: TaskFlow . . . 5

Prerequisites . . . 5

Decorators: the one TypeScript feature you must understand . . . 5

**Episode 1: What is NestJS and Why Does It Exist? . . . 7**

The problem NestJS solves . . . 7

Express vs Fastify vs NestJS . . . 8

Three big ideas . . . 8

When to choose NestJS (and when not to) . . . 8

**Episode 2: Setup and Project Tour . . . 9**

Install and create the project . . . 9

main.ts: the front door . . . 9

npm scripts you will use daily . . . 9

A local PostgreSQL in one command . . . 10

**Episode 3: The Holy Trinity: Modules, Controllers, Providers . . . 11**

Generate the Tasks feature . . . 11

The provider (service) . . . 11

The controller . . . 12

The module . . . 13

Modules and visibility . . . 13

**Episode 4: Dependency Injection: The Heart of Nest . . . 15**

The problem DI solves . . . 15

How the IoC container works . . . 15

Provider scopes . . . 16

Custom providers . . . 16

Global and dynamic modules . . . 16

**Episode 5: The Request Lifecycle . . . 18**

Worked example: a bad request that has a valid token . . . 19

Order inside each stage . . . 19

**Episode 6: Controllers in Depth . . . 20**

The decorator vocabulary . . . 20

Pagination and filtering with query parameters . . . 20

Status codes and empty responses . . . 21

Controller rules . . . 21
Hiding fields from responses . . . 21

**Episode 7: DTOs and Validation . . . 22**

Theory: what is a DTO? . . . 22
ValidationPipe options . . . 22
Common validators . . . 23
A richer DTO: enums, nested objects and query DTOs . . . 23
Reusing DTOs: mapped types . . . 23

**Episode 8: A Real Database with TypeORM and PostgreSQL . . . 24**

Configuration first . . . 24
Entities: classes that map to tables . . . 25
Register the entity and inject the repository . . . 25
Repository methods you will use most . . . 26
Transactions . . . 26

**Episode 9: Authentication with JWT and Guards . . . 27**

Theory: authentication vs authorization . . . 27
The users service . . . 28
The auth service and controller . . . 28
The auth module . . . 29
The guard . . . 29
A custom parameter decorator . . . 29
Protecting the Tasks controller . . . 30
Make everything protected by default . . . 30
Role-based access control . . . 30
Security checklist for auth . . . 31

**Episode 10: Interceptors, Filters and Middleware . . . 32**

Interceptors . . . 32
Exception filters . . . 32
Middleware . . . 33
Where can each tool be attached? . . . 33

**Episode 11: Testing Like a Professional . . . 34**

Unit test: a service with a fake repository . . . 34
End-to-end test . . . 35
Testing tips . . . 35

**Episode 12: API Docs, Config and Production Polish . . . 36**

Swagger / OpenAPI documentation . . . 36

Validate environment variables at startup . . . 36
Security and robustness . . . 36
Logging, health and containers . . . 36
Pre-launch checklist . . . 37

**Episode 13: Beyond REST: The Hero Tier . . . 38**

WebSockets: live task updates . . . 39
Background jobs with a queue . . . 39
Scheduling and events . . . 39
Microservices . . . 39
Monolith first, microservices later . . . 40
GraphQL in one glance . . . 40

**Episode 14: Architecture, Best Practices and Troubleshooting . . . 41**

Organize by feature, not by type . . . 41
Rules I live by . . . 41
Common errors and how to fix them . . . 42
Performance notes . . . 42

**Episode 15: Your Roadmap and Practice Challenges . . . 43**

Practice challenges . . . 43
Where to learn more . . . 43

**Appendix A: Cheat Sheets . . . 44**

Nest CLI . . . 44
Decorator quick reference . . . 44
Built-in HTTP exceptions . . . 45
Built-in pipes . . . 45

**Appendix B: Glossary . . . 46**

EPISODE 0

# Welcome and the Plan

*Before we write a single line, let us agree on what we are building and how to use this guide.*

Hey everyone, welcome. This guide is the full course I would give you if we were sitting together at a whiteboard. Every episode follows the same rhythm: **theory first** (what the thing is and why it exists), then **code** (we build it), then a **diagram** so the idea sticks. By the end you will understand NestJS the way a framework author does: not as a pile of decorators, but as one coherent architecture.

## What we are building: TaskFlow

One project runs through the whole guide: **TaskFlow**, a task-manager REST API where each user manages their own tasks. It is small enough to finish and big enough to need everything real apps need: validation, database, authentication, error handling, tests.

| Method | Route | What it does | Auth |
|---|---|---|---|
| POST | `/auth/register` | Create an account | Public |
| POST | `/auth/login` | Exchange credentials for a JWT | Public |
| GET | `/tasks` | List my tasks (paginated) | JWT |
| GET | `/tasks/:id` | Get one of my tasks | JWT |
| POST | `/tasks` | Create a task | JWT |
| PATCH | `/tasks/:id` | Update a task | JWT |
| DELETE | `/tasks/:id` | Delete a task | JWT |

## Prerequisites

- **JavaScript and TypeScript basics**: classes, async/await, modules, types and interfaces.
- **HTTP and REST**: methods, status codes, headers, JSON bodies.
- **Basic SQL** (helpful, not mandatory) and **Docker** if you want a local PostgreSQL in one command.
- **Node.js** (use a current LTS version) and a code editor. VS Code works great.

## Decorators: the one TypeScript feature you must understand

NestJS is built on **decorators**. A decorator is just a function that receives information about the thing it is attached to (a class, a method, a parameter) and may attach extra information or change it. Seeing one written by hand removes the magic:

decorator-demo.ts

```ts
function Log(target: any, key: string, desc: PropertyDescriptor) {
  const original = desc.value;
  desc.value = function (...args: any[]) {
    console.log(`Calling ${key} with`, args);   // runs before the real method
    return original.apply(this, args);
  };
}

class Calculator {
  @Log
  add(a: number, b: number) {
    return a + b;
  }
}

new Calculator().add(2, 3);   // logs: Calling add with [2, 3]  -> returns 5
```

Nest decorators like `@Controller()`, `@Get()` and `@Injectable()` mostly do one job: **store metadata** using the `reflect-metadata` library. At startup Nest reads that metadata and wires your application together. Two `tsconfig` flags make this work, and the Nest CLI sets both for you:

- `experimentalDecorators: true`: enables decorator syntax.
- `emitDecoratorMetadata: true`: makes TypeScript emit the **types** of constructor parameters. This is how Nest knows that `constructor(private svc: TasksService)` needs a `TasksService`. Remember this; it is the key to dependency injection.

> **INSTRUCTOR TIP**
> Do not try to memorize every decorator. Learn the **five building blocks** (module, controller, provider, pipe/guard/interceptor/filter, DTO) and the decorators become obvious vocabulary.

EPISODE 1

# What is NestJS and Why Does It Exist?

*Express gives you freedom. NestJS gives you architecture.*

## The problem NestJS solves

Express is a minimal tool: `app.get('/x', handler)` and you are on your own. At 200 lines that is wonderful. At 20,000 lines with six developers, every person invents their own folder layout, their own way to share a database connection, their own validation style. Nothing is wrong with Express; it simply does not answer the question *how should a large application be organized?*

Angular answered that question for the frontend with modules, dependency injection and decorators. **NestJS applies the same ideas to the backend.** If you have seen Spring (Java) or ASP.NET Core (C#), the concepts will feel familiar.

> **THEORY**
> **NestJS** is an opinionated, TypeScript-first framework that runs on top of Express (default) or Fastify and gives you modules, dependency injection, decorators and a standard request lifecycle.

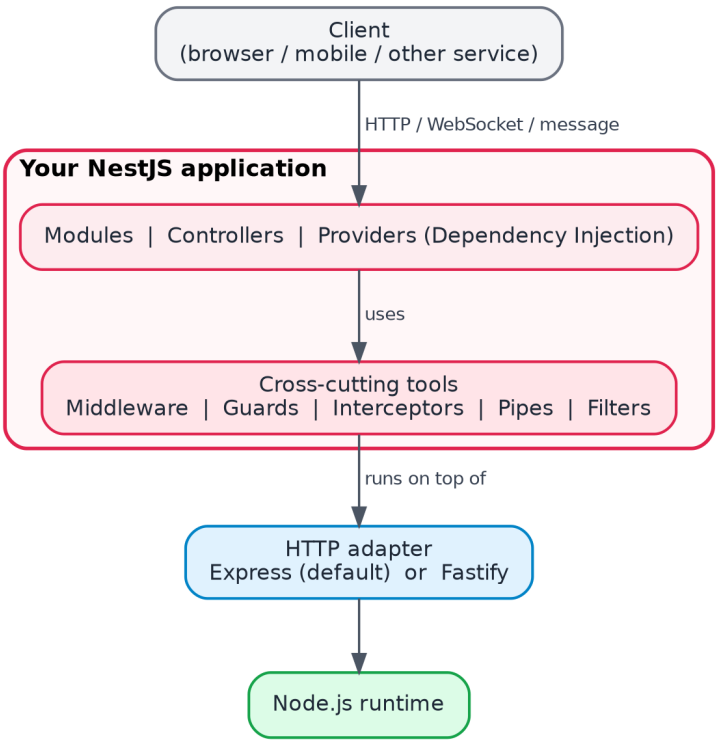



*Figure 1. NestJS sits between your code and the HTTP server. It does not replace Express or Fastify; it organizes your app on top of them.*

## Express vs Fastify vs NestJS

| | Express | Fastify | NestJS |
|---|---|---|---|
| What it is | Minimal HTTP library | Minimal, fast HTTP library | Application framework |
| Structure | You decide | You decide (plugins) | Modules, controllers, providers |
| Dependency injection | None built in | None built in | Built in, first class |
| Validation, auth, docs | Pick libraries yourself | Pick libraries yourself | Official, integrated packages |
| TypeScript | Optional | Optional | Default and expected |
| Best for | Small APIs, prototypes | High-throughput services | Medium to large apps, teams |

## Three big ideas

1. **Decorators** attach metadata to classes and methods (`@Controller`, `@Get`, `@Injectable`).
2. **Dependency Injection** means you never write `new TasksService()`. You declare what you need in the constructor and Nest supplies it.
3. **Modules** are boxes that group related code and declare what they share with other boxes.

## When to choose NestJS (and when not to)

- **Choose it** when the codebase will grow, when several developers share it, or when you want consistent patterns for REST, GraphQL, WebSockets and microservices in one framework.
- **Think twice** for a 50-line webhook receiver or a throwaway script. A tiny Express or Fastify file is simpler there.

EPISODE 2

# Setup and Project Tour

*Install the CLI, generate a project, and understand every file Nest creates.*

## Install and create the project

terminal

```
npm i -g @nestjs/cli          # the nest command
nest new taskflow             # choose npm when asked
cd taskflow
npm run start:dev             # watch mode with automatic reload
```

Open `http://localhost:3000` and you will see *Hello World!*. Congratulations, you have a running Nest server. Here is what was generated:

project layout

```
taskflow/
|-- src/
|   |-- main.ts                    entry point, boots the application
|   |-- app.module.ts              the root module
|   |-- app.controller.ts          sample controller (routes)
|   |-- app.service.ts             sample provider (logic)
|   `-- app.controller.spec.ts     sample unit test
|-- test/                          end-to-end tests
|-- nest-cli.json                  CLI configuration
|-- tsconfig.json                  TypeScript configuration
`-- package.json
```

## main.ts: the front door

src/main.ts

```ts
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);   // builds the whole dependency graph
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` starts from the root module, discovers every module it imports, instantiates every provider, and registers every route. We will unpack `ValidationPipe` in Episode 7.

## npm scripts you will use daily

| Script | Purpose |
|---|---|
| `npm run start:dev` | Run with file watching (use this while developing) |
| `npm run build` | Compile TypeScript into `dist/` |
| `npm run start:prod` | Run the compiled build with `node dist/main` |
| `npm test` | Run unit tests (Jest) |
| `npm run test:e2e` | Run end-to-end tests (Jest + supertest) |
| `npm run test:cov` | Unit tests with a coverage report |
| `npm run lint` | ESLint check |

## A local PostgreSQL in one command

We will use PostgreSQL from Episode 8. Create this file in the project root and run `docker compose up -d`:

**docker-compose.yml**

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: taskflow
      POSTGRES_PASSWORD: taskflow
      POSTGRES_DB: taskflow
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

**.env (never commit this file: add it to .gitignore)**

```
DATABASE_URL=postgres://taskflow:taskflow@localhost:5432/taskflow
JWT_SECRET=change-me-to-a-long-random-string
PORT=3000
```

**INSTRUCTOR TIP**

The CLI is your best friend. `nest g resource tasks` generates a module, controller, service, DTOs and spec files in one go. Use `nest g --help` to see every generator (module, controller, service, guard, pipe, interceptor, filter, decorator...).

EPISODE 3

# The Holy Trinity: Modules, Controllers, Providers

*Three building blocks explain 80 percent of every Nest application.*

Think of a restaurant. The **controller** is the waiter: it takes the request, hands back the response, and never cooks. The **provider** (usually a *service*) is the chef: all the real logic lives there. The **module** is the section of the restaurant that groups waiters and chefs who work together.

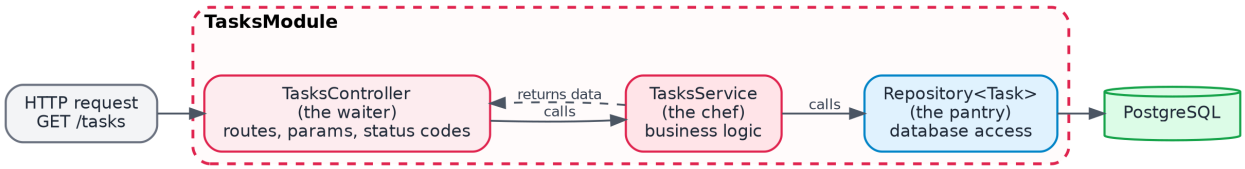



*Figure 2. A request flows controller to service to repository to database. Each layer has exactly one job.*

## Generate the Tasks feature

```
nest g resource tasks   # choose: REST API, and yes to CRUD entry points
```

## The provider (service)

We start with an in-memory array so you can focus on structure. In Episode 8 we swap in a real database and you will see that almost nothing else changes.

src/tasks/tasks.service.ts

```ts
import { Injectable, NotFoundException } from '@nestjs/common';

export interface Task {
  id: number;
  title: string;
  done: boolean;
}

@Injectable()                       // "Nest, you are allowed to inject this class"
export class TasksService {
  private tasks: Task[] = [];
  private nextId = 1;

  findAll() {
    return this.tasks;
  }

  findOne(id: number) {
    const task = this.tasks.find((t) => t.id === id);
    if (!task) throw new NotFoundException(`Task ${id} not found`);   // becomes HTTP 404
    return task;
  }

  create(title: string) {
    const task: Task = { id: this.nextId++, title, done: false };
    this.tasks.push(task);
    return task;
  }

  remove(id: number) {
    const task = this.findOne(id);
    this.tasks = this.tasks.filter((t) => t.id !== id);
    return task;
  }
}
```

# The controller

src/tasks/tasks.controller.ts

```ts
import { Body, Controller, Delete, Get, Param, ParseIntPipe, Post } from '@nestjs/common';
import { TasksService } from './tasks.service';

@Controller('tasks')                                  // every route starts with /tasks
export class TasksController {
  constructor(private readonly tasksService: TasksService) {}   // dependency injection

  @Get()                                              // GET /tasks
  findAll() {
    return this.tasksService.findAll();
  }

  @Get(':id')                                         // GET /tasks/5
  findOne(@Param('id', ParseIntPipe) id: number) {    // "5" (string) -> 5 (number)
    return this.tasksService.findOne(id);
  }

  @Post()                                             // POST /tasks
  create(@Body() body: { title: string }) {
    return this.tasksService.create(body.title);
  }

  @Delete(':id')                                      // DELETE /tasks/5
  remove(@Param('id', ParseIntPipe) id: number) {
    return this.tasksService.remove(id);
  }
}
```

## The module

src/tasks/tasks.module.ts

```
import { Module } from '@nestjs/common';
import { TasksController } from './tasks.controller';
import { TasksService } from './tasks.service';

@Module({
  controllers: [TasksController],
  providers: [TasksService],
  exports: [TasksService],        // allow OTHER modules to use this service
})
export class TasksModule {}
```

src/app.module.ts

```
@Module({
  imports: [TasksModule],         // the root module imports the feature module
})
export class AppModule {}
```

Try it from a second terminal:

terminal

```
curl -X POST localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Learn Nest"}'
curl localhost:3000/tasks
curl localhost:3000/tasks/1
curl -X DELETE localhost:3000/tasks/1
```

## Modules and visibility

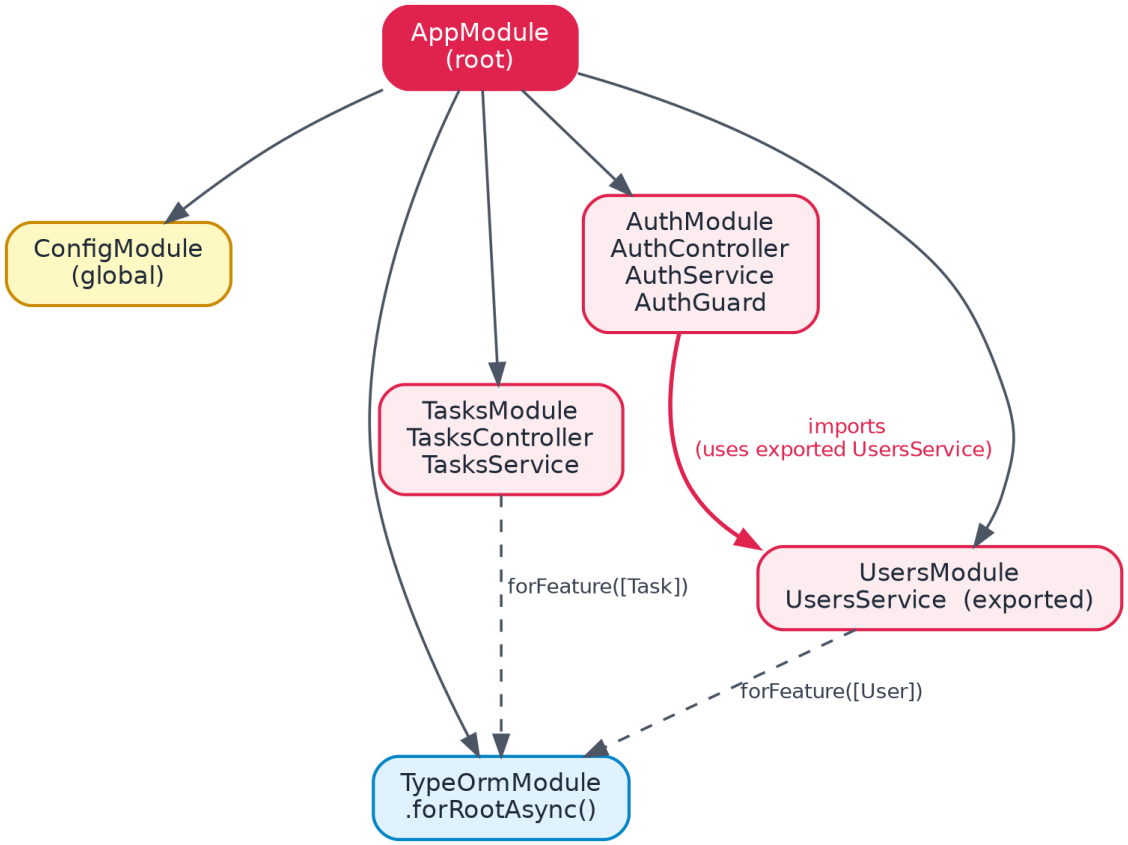



*Figure 3. The module graph of TaskFlow. AuthModule can use UsersService only because UsersModule exports it and AuthModule imports UsersModule.*

**WATCH OUT**

**Golden rule:** a provider is private to its module unless that module lists it in `exports` AND the consuming module lists the module in `imports`. The error "*Nest can't resolve dependencies of X*" is, nine times out of ten, a violation of this rule.

EPISODE 4

# Dependency Injection: The Heart of Nest

*Why does constructor(private svc: TasksService) just work?*

## The problem DI solves

Imagine a controller that creates its own dependencies:

```
the painful way (do not do this)
class TasksController {
  // tightly coupled
  private service = new TasksService(new Repository(new Database(new Config())));
}
```

This code is hard to test (how do you replace the real database with a fake?), hard to change (every place that builds a service must be edited), and wasteful (a new `Database` for every controller?). **Dependency Injection** inverts the responsibility: a class only *declares* what it needs, and an external container *provides* it.

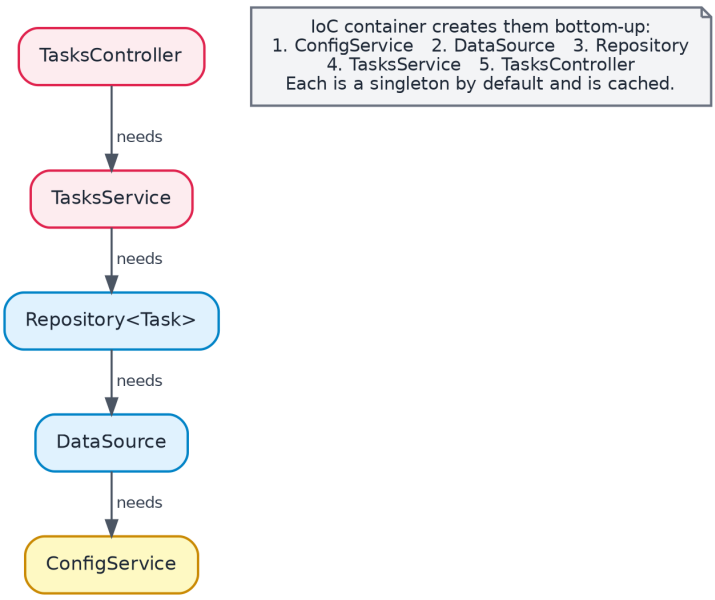



*Figure 4. Nest reads constructor types, builds the dependency tree, and creates instances bottom-up as singletons.*

## How the IoC container works

1. **Scan**: Nest walks the module graph from `AppModule` and collects every provider.
2. **Read metadata**: for each class it reads the constructor parameter types emitted by TypeScript.
3. **Resolve**: it builds the dependency tree and detects problems (missing provider, circular dependency).
4. **Instantiate**: it creates instances from the leaves up and caches them.
5. **Inject**: when a controller or service is created, its constructor receives the cached instances.

## Provider scopes

| Scope | Lifetime | When to use |
|---|---|---|
| `DEFAULT` (singleton) | One instance for the whole application | Almost always. Fast and simple. |
| `REQUEST` | A new instance for every incoming request | Per-request state (e.g. multi-tenancy). Slower, and it makes everything depending on it request-scoped too. |
| `TRANSIENT` | A new instance for every class that injects it | Rare. Each consumer needs its own private copy. |

changing the scope

```
@Injectable({ scope: Scope.REQUEST })
export class RequestContextService { /* one per HTTP request */ }
```

## Custom providers

A class is only one way to provide something. The long-form syntax lets you provide values, swap implementations, or build objects with logic:

provider flavours

```
@Module({
  providers: [
    // 1. useValue: constants and test doubles
    { provide: 'APP_NAME', useValue: 'TaskFlow' },

    // 2. useClass: choose an implementation per environment
    {
      provide: MailService,
      useClass: process.env.NODE_ENV === 'test' ? FakeMailService : SmtpMailService,
    },

    // 3. useFactory: build with logic, possibly async, using other providers
    {
      provide: 'REDIS',
      useFactory: async (config: ConfigService) => createRedis(config.get('REDIS_URL')),
      inject: [ConfigService],
    },

    // 4. useExisting: an alias for another provider
    { provide: 'MAIL', useExisting: MailService },
  ],
})
export class NotificationsModule {}
```

Non-class providers are injected by a **token** (usually a string or a `Symbol`):

```
constructor(@Inject('APP_NAME') private readonly appName: string) {}
```

## Global and dynamic modules

- A module decorated with `@Global()` exports its providers to every module without explicit imports. Use sparingly (config and logging are good candidates).
- A **dynamic module** is configured when imported: `ConfigModule.forRoot({...})`, `JwtModule.registerAsync({...})`, `TypeOrmModule.forFeature([Task])`. These static methods return a module definition built from your options. You will use them constantly and can write your own.
- `@Optional()` marks a dependency that may be absent; Nest will inject `undefined` instead of failing.

> **WATCH OUT**
> **Circular dependencies** (A needs B, B needs A) are a design smell. `forwardRef(() => B)` silences the error, but the better fix is usually to extract the shared logic into a third provider that both can use.

EPISODE 5

# The Request Lifecycle

*If you memorize one diagram from this guide, make it this one.*

Every HTTP request travels through the same pipeline. Knowing the order answers most debugging questions: *why did my guard run before validation?*, *why did my filter not catch this?*, *why is my interceptor seeing the response twice?*

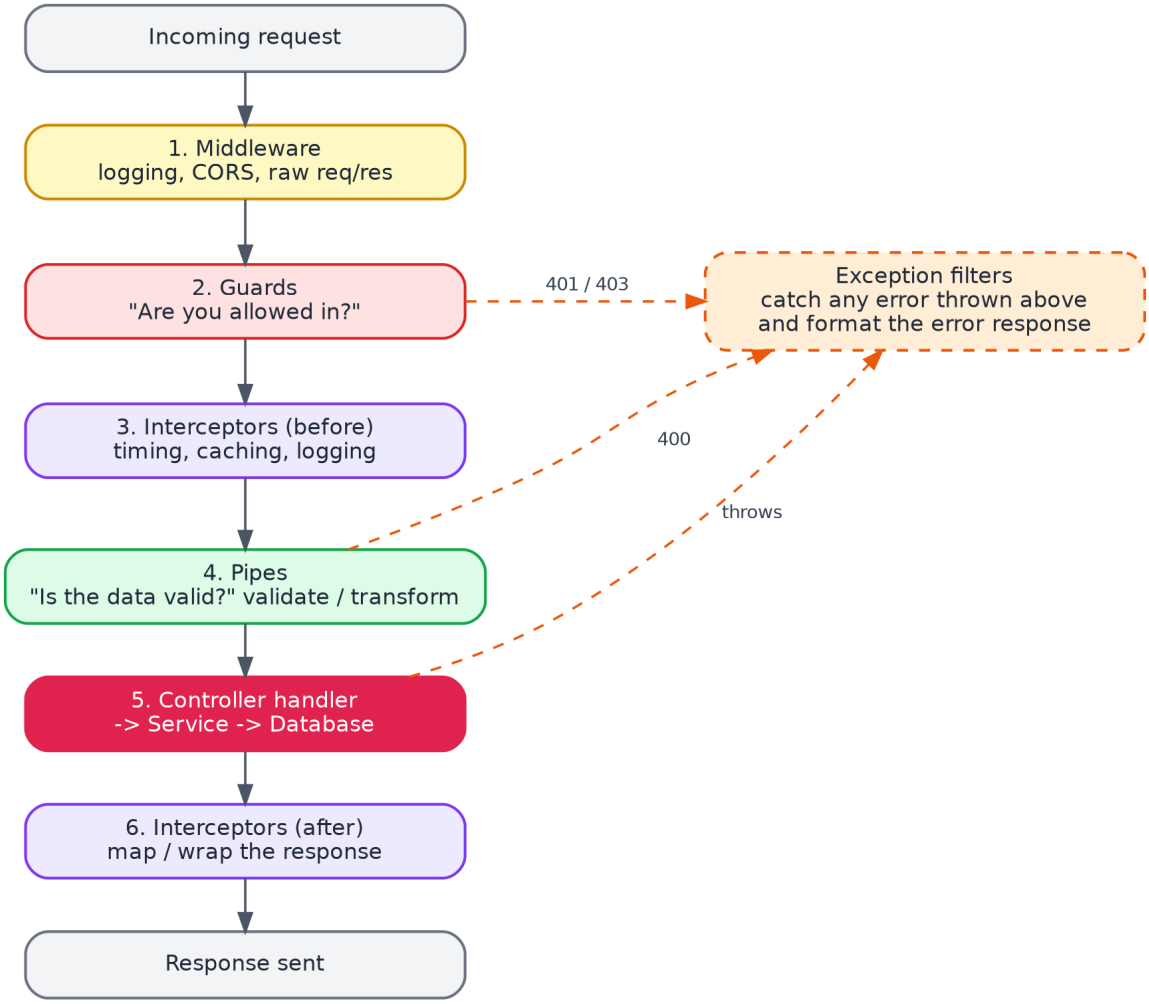



*Figure 5. The full request lifecycle. Dashed lines show that exception filters catch an error thrown at any stage.*

| Tool | Question it answers | Typical use | Built-in example |
|---|---|---|---|
| Middleware | Do something with the raw request | Logging, CORS, body parsing | `helmet()`, custom logger |
| Guard | Is this caller allowed to proceed? | Authentication, roles | `AuthGuard`, `RolesGuard` |
| Interceptor | Wrap or transform around the handler | Timing, caching, response mapping | `ClassSerializerInterceptor` |
| Pipe | Is the input valid? Convert it | Validation, parsing | `ValidationPipe`, `ParseIntPipe` |

| Tool | Question it answers | Typical use | Built-in example |
|---|---|---|---|
| Exception filter | How should errors look? | Uniform error JSON | Default `HttpException` handler |

**INSTRUCTOR TIP**
Memory trick: **M**y **G**uard **I**s **P**retty **C**ool, **I**nterceptors, **F**ilters. (Middleware, Guards, Interceptors, Pipes, Controller, Interceptors again, Filters.)

## Worked example: a bad request that has a valid token

A client sends `POST /tasks` with a valid JWT and the body `{"title": ""}`. Which components run, and in what order?

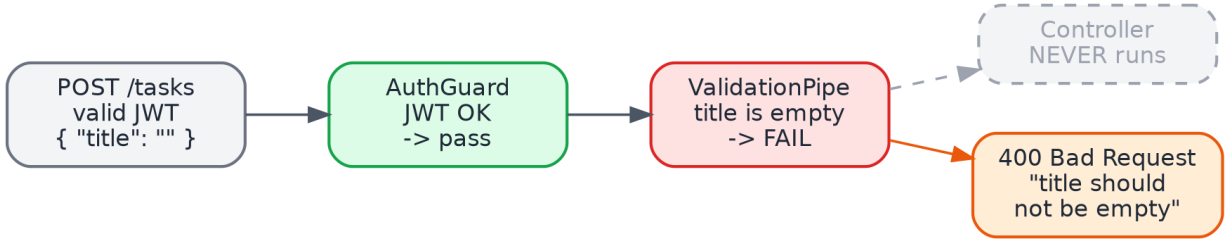



*Figure 6. The guard passes (the token is fine), then the validation pipe rejects the body. The controller never executes.*

| Step | Component | Result |
|---|---|---|
| 1 | Middleware | Logs the request and calls `next()` |
| 2 | `AuthGuard` | Token verified, `req.user` set, returns `true` |
| 3 | Interceptors (before) | Timer starts |
| 4 | `ValidationPipe` | `title` fails `@IsNotEmpty()`, throws `BadRequestException` |
| 5 | Controller | **Never runs**. Neither does the service. |
| 6 | Exception filter | Formats the 400 response with the validation messages |

Rule of thumb: **guards run before pipes**. We check *who you are* before we spend effort checking *what you sent*.

## Order inside each stage

- Guards and interceptors run **global, then controller, then route**.
- Pipes run **global, then controller, then route, then parameter**.
- Exception filters work the other way round: the most specific one (route) gets the first chance to catch the error, then controller, then global.
- Interceptors wrap each other like an onion: the first to start is the last to finish on the way out.

EPISODE 6

# Controllers in Depth

*Routing, parameters, status codes, pagination and shaping the response.*

## The decorator vocabulary

| Decorator | Purpose | Example |
|---|---|---|
| `@Controller('tasks')` | Class: base route prefix | `/tasks/...` |
| `@Get() @Post() @Put() @Patch() @Delete()` | Method: HTTP verb and sub-path | `@Get(':id')` |
| `@Param('id')` | URL path parameter | `/tasks/5` gives `'5'` |
| `@Query('page')` | Query string value | `?page=2` |
| `@Body()` | Parsed JSON body (use a DTO class) | `@Body() dto: CreateTaskDto` |
| `@Headers('authorization')` | A request header | `Bearer abc...` |
| `@Req() / @Res()` | Raw platform objects (avoid when possible) | Ties code to Express/Fastify |
| `@HttpCode(204)` | Override the response status | Delete with no body |
| `@Header('Cache-Control','none')` | Set a response header | |
| `@Redirect(url, 301)` | Redirect | |

## Pagination and filtering with query parameters

Query strings arrive as strings. Pipes such as `ParseIntPipe` and `DefaultValuePipe` convert them and supply defaults before your handler runs:

pagination in the controller

```
import { DefaultValuePipe, ParseBoolPipe, ParseIntPipe, Query } from '@nestjs/common';

@Get()
findAll(
  @Query('page', new DefaultValuePipe(1), ParseIntPipe) page: number,
  @Query('limit', new DefaultValuePipe(10), ParseIntPipe) limit: number,
  @Query('done', new ParseBoolPipe({ optional: true })) done?: boolean,
) {
  // GET /tasks?page=2&limit=5&done=true
  return this.tasksService.findAll({ page, limit, done });
}
```

## Status codes and empty responses

| Situation | Status | How in Nest |
|---|---|---|
| Successful GET / PATCH | 200 | default for GET, PATCH, PUT, DELETE |
| Successful POST | 201 | default for POST |
| Success with no body | 204 | `@HttpCode(204)` and return nothing |
| Invalid input | 400 | `BadRequestException` / `ValidationPipe` |
| Not logged in | 401 | `UnauthorizedException` |
| Logged in but not allowed | 403 | `ForbiddenException` |
| Missing resource | 404 | `NotFoundException` |
| Duplicate / conflict | 409 | `ConflictException` |

a 204 response

```
@Delete(':id')
@HttpCode(204)
async remove(@Param('id', ParseIntPipe) id: number) {
  await this.tasksService.remove(id);   // returning nothing sends an empty 204
}
```

## Controller rules

- **Route order matters.** Put static routes (`@Get('stats')`) **before** parameterized ones (`@Get(':id')`), or `/tasks/stats` will be captured as `id = 'stats'`.
- **Prefer returning values** over using `@Res()`. If you inject `@Res()` you take manual control of the response and lose interceptors and automatic serialization (unless `passthrough: true`).
- **Async is fine.** Return a Promise or use `async`; Nest awaits it for you.
- **Versioning:** `app.enableVersioning({ type: VersioningType.URI })` plus `@Controller({ path: 'tasks', version: '1' })` gives you `/v1/tasks`.
- **Global prefix:** `app.setGlobalPrefix('api')` turns every route into `/api/...`.

## Hiding fields from responses

Never return entities that contain secrets (like a password hash). Mark fields with `@Exclude()` and enable `ClassSerializerInterceptor`; Nest will strip them on the way out:

serialization

```
import { Exclude } from 'class-transformer';

export class UserResponse {
  id: number;
  email: string;

  @Exclude()
  passwordHash: string;          // never serialized
}

// in main.ts
app.useGlobalInterceptors(new ClassSerializerInterceptor(app.get(Reflector)));
```

> **GOOD TO KNOW**
> The serializer only works on **class instances**, not plain objects. If you return raw objects from the database layer, map them to a response class (or select only the columns you need).

EPISODE 7

# DTOs and Validation

*Never trust the client. TypeScript types vanish at runtime, so validate for real.*

## Theory: what is a DTO?

A **Data Transfer Object** describes the shape of data crossing a boundary: what the client may send, or what you return. In Episode 3 we typed the body as `{ title: string }`. That protects you from nothing, because TypeScript interfaces are erased when the code compiles and the JSON from the network can contain anything. A DTO must be a **class** (not an interface) so decorators and metadata survive at runtime.

terminal

```
npm i class-validator class-transformer
```

src/tasks/dto/create-task.dto.ts

```
import { IsBoolean, IsNotEmpty, IsOptional, IsString, MaxLength } from 'class-validator';

export class CreateTaskDto {
  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  title: string;

  @IsOptional()
  @IsBoolean()
  done?: boolean;
}
```

Use it in the controller: `create(@Body() dto: CreateTaskDto)`. Because `ValidationPipe` is registered globally in `main.ts`, an empty title now produces:

automatic 400 response

```
{
  "message": ["title should not be empty"],
  "error": "Bad Request",
  "statusCode": 400
}
```

## ValidationPipe options

| Option | Effect |
|---|---|
| `whitelist: true` | Silently **strips** properties that have no decorator in the DTO. Blocks mass-assignment (a client sending `role: 'admin'`). |
| `forbidNonWhitelisted: true` | With `whitelist`, **rejects** the request instead of stripping. |
| `transform: true` | Converts the plain payload into a DTO instance and converts primitives (`'5'` to `5`) based on the declared types. |
| `disableErrorMessages: true` | Hides detailed messages (some teams do this in production). |
| `stopAtFirstError: true` | Reports only the first failing rule per property. |

## Common validators

| Decorator | Checks |
|---|---|
| `@IsString()` `@IsInt()` `@IsNumber()` `@IsBoolean()` `@IsDate()` | Type |
| `@IsEmail()` `@IsUrl()` `@IsUUID()` | Format |
| `@MinLength(8)` `@MaxLength(100)` | String length |
| `@Min(1)` `@Max(100)` | Number range |
| `@IsEnum(Priority)` | Value is one of an enum |
| `@IsArray()` `@ArrayMinSize(1)` | Arrays |
| `@IsOptional()` | Skip other validators when the value is `undefined` or `null` |
| `@ValidateNested()` `@Type(() => ChildDto)` | Validate a nested object or array of objects |

## A richer DTO: enums, nested objects and query DTOs

dto/create-task.dto.ts (extended)

```
import { Type } from 'class-transformer';
import { IsEnum, IsInt, IsOptional, Max, Min, ValidateNested } from 'class-validator';

export enum Priority { LOW = 'low', MEDIUM = 'medium', HIGH = 'high' }

class ReminderDto {
  @IsInt() @Min(1) minutesBefore: number;
}

export class CreateTaskDto {
  // ...title as before
  @IsEnum(Priority) priority: Priority = Priority.MEDIUM;

  @IsOptional()
  @ValidateNested()
  @Type(() => ReminderDto)        // required so nested validation can find the class
  reminder?: ReminderDto;
}

export class ListTasksQuery {
  @IsOptional() @Type(() => Number) @IsInt() @Min(1) page = 1;
  @IsOptional() @Type(() => Number) @IsInt() @Min(1) @Max(100) limit = 10;
}
// controller: findAll(@Query() query: ListTasksQuery)
```

## Reusing DTOs: mapped types

Writing a second DTO for updates is boring and error-prone. Mapped types derive it:

src/tasks/dto/update-task.dto.ts

```
import { PartialType } from '@nestjs/mapped-types';   // or '@nestjs/swagger' if you use Swagger

export class UpdateTaskDto extends PartialType(CreateTaskDto) {}   // every field becomes optional

// Also available: PickType(Dto, ['title']), OmitType(Dto, ['done']), IntersectionType(A, B)
```

**GOOD TO KNOW**

If you use Swagger, import `PartialType`, `PickType` and friends from `@nestjs/swagger` instead. They behave the same but also keep the API documentation metadata in sync.

EPISODE 8

# A Real Database with TypeORM and PostgreSQL

*Swap the in-memory array for persistent storage without rewriting the architecture.*

terminal

```
npm i @nestjs/typeorm typeorm pg @nestjs/config
```

## Configuration first

Never hard-code credentials. `@nestjs/config` loads `.env` and exposes values through an injectable `ConfigService`. Because TypeORM needs that config, we use the **async** registration style (a factory that receives injected dependencies):

src/app.module.ts

```
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),            // reads .env, available everywhere
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'postgres',
        url: config.get<string>('DATABASE_URL'),
        autoLoadEntities: true,      // entities registered with forFeature() are picked up
        synchronize: false,          // see the warning below
      }),
    }),
    TasksModule,
    UsersModule,
    AuthModule,
  ],
})
export class AppModule {}
```

**WATCH OUT**

`synchronize: true` makes TypeORM alter your tables to match your entities on every start. It is convenient in a throwaway dev database and **dangerous anywhere else** (it can drop columns and data). For real projects use **migrations**.

## Entities: classes that map to tables

user.entity.ts and task.entity.ts (shown together)

```
import { Column, CreateDateColumn, Entity, ManyToOne, OneToMany,
         PrimaryGeneratedColumn } from 'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn() id: number;
  @Column({ unique: true }) email: string;
  @Column() passwordHash: string;
  @Column({ default: 'user' }) role: string;
  @OneToMany(() => Task, (task) => task.owner) tasks: Task[];
}

@Entity()
export class Task {
  @PrimaryGeneratedColumn() id: number;
  @Column() title: string;
  @Column({ default: false }) done: boolean;
  @Column() ownerId: number;
  @ManyToOne(() => User, (user) => user.tasks, { onDelete: 'CASCADE' }) owner: User;
  @CreateDateColumn() createdAt: Date;
}
```

## Register the entity and inject the repository

tasks.module.ts

```
@Module({
  imports: [TypeOrmModule.forFeature([Task])],       // creates Repository<Task> provider
  controllers: [TasksController],
  providers: [TasksService],
})
export class TasksModule {}
```

tasks.service.ts (database version)

```
@Injectable()
export class TasksService {
  constructor(@InjectRepository(Task) private readonly repo: Repository<Task>) {}

  async findAll(ownerId: number, page = 1, limit = 10) {
    const [items, total] = await this.repo.findAndCount({
      where: { ownerId },
      order: { createdAt: 'DESC' },
      skip: (page - 1) * limit,
      take: limit,
    });
    return { items, total, page, limit };
  }

  async findOne(id: number, ownerId: number) {
    const task = await this.repo.findOne({ where: { id, ownerId } });
    if (!task) throw new NotFoundException(`Task ${id} not found`);
    return task;
  }

  create(dto: CreateTaskDto, ownerId: number) {
    return this.repo.save(this.repo.create({ ...dto, ownerId }));
  }

  async update(id: number, dto: UpdateTaskDto, ownerId: number) {
    const task = await this.findOne(id, ownerId);   // also enforces ownership
    return this.repo.save({ ...task, ...dto });
  }

  async remove(id: number, ownerId: number) {
    await this.repo.remove(await this.findOne(id, ownerId));
  }
}
```

Notice that the **service kept the same shape**. The controller does not care whether data comes from an array or PostgreSQL. That is the payoff of layering.

## Repository methods you will use most

| Method | What it does |
|---|---|
| `find(options)` / `findOne(options)` | Read rows with `where`, `order`, `relations`, `select` |
| `findAndCount(options)` | Rows plus total count (ideal for pagination) |
| `create(partial)` | Build an entity instance (does not save) |
| `save(entity)` | Insert or update (upsert by primary key) |
| `update(criteria, partial)` | Fast partial update without loading |
| `delete(criteria)` / `remove(entity)` | Delete by criteria / delete a loaded entity |
| `createQueryBuilder('t')` | Build custom SQL fluently (joins, aggregates) |

## Transactions

When two writes must succeed or fail together, run them in a transaction:

transaction example

```
constructor(private readonly dataSource: DataSource) {}

async archiveAndLog(task: Task) {
  await this.dataSource.transaction(async (manager) => {
    await manager.save(Task, { ...task, done: true });
    await manager.save(AuditLog, { action: 'archive', taskId: task.id });
  });                          // if either save throws, both are rolled back
}
```

**GOOD TO KNOW**

Prefer Prisma or MikroORM? Both integrate well with Nest. The architecture is identical: only the repository layer inside your services changes. Wrap the client in a provider (for Prisma, a `PrismaService`) and inject it exactly as you inject any other service.

EPISODE 9

# Authentication with JWT and Guards

*Register, log in, receive a token, and protect routes.*

## Theory: authentication vs authorization

- **Authentication** answers *who are you?* (login, token verification).
- **Authorization** answers *what may you do?* (roles, ownership checks).
- A **JWT** (JSON Web Token) is a signed string containing claims (such as the user id). The server can verify the signature without storing sessions, so any request carrying a valid token is trusted to be that user until it expires.

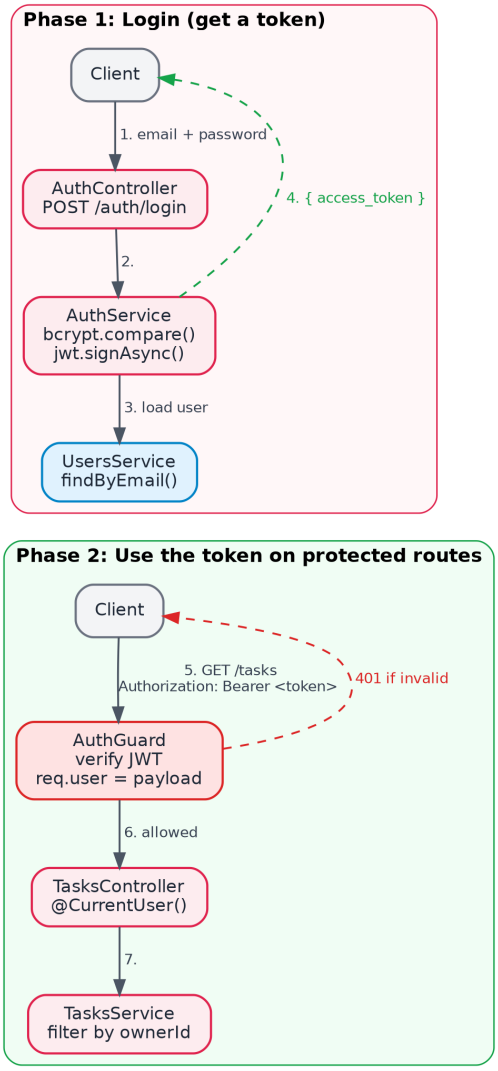



*Figure 7. Phase 1 exchanges credentials for a token. Phase 2 sends the token with each request and the guard verifies it.*

terminal

```
npm i @nestjs/jwt bcrypt
npm i -D @types/bcrypt
nest g module auth && nest g service auth && nest g controller auth
nest g module users && nest g service users
```

## The users service

users.service.ts and users.module.ts

```
@Injectable()
export class UsersService {
  constructor(@InjectRepository(User) private readonly repo: Repository<User>) {}

  findByEmail(email: string) {
    return this.repo.findOne({ where: { email } });
  }

  async create(email: string, passwordHash: string) {
    if (await this.findByEmail(email)) throw new ConflictException('Email already registered');
    return this.repo.save(this.repo.create({ email, passwordHash }));
  }
}

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService],              // AuthModule needs this
})
export class UsersModule {}
```

## The auth service and controller

auth.service.ts

```
@Injectable()
export class AuthService {
  constructor(private readonly users: UsersService, private readonly jwt: JwtService) {}

  async register(email: string, password: string) {
    const passwordHash = await bcrypt.hash(password, 10);   // NEVER store plain passwords
    const user = await this.users.create(email, passwordHash);
    return { id: user.id, email: user.email };              // do not leak the hash
  }

  async login(email: string, password: string) {
    const user = await this.users.findByEmail(email);
    const ok = user && (await bcrypt.compare(password, user.passwordHash));
    // same message for unknown email and wrong password
    if (!user || !ok) throw new UnauthorizedException('Invalid credentials');
    const payload = { sub: user.id, email: user.email, role: user.role };
    return { access_token: await this.jwt.signAsync(payload) };
  }
}
```

auth.dto.ts and auth.controller.ts

```
export class AuthDto {
  @IsEmail() email: string;
  @IsString() @MinLength(8) @MaxLength(72) password: string;   // bcrypt ignores bytes past 72
}

@Controller('auth')
export class AuthController {
  constructor(private readonly auth: AuthService) {}

  @Post('register')
  register(@Body() dto: AuthDto) {
    return this.auth.register(dto.email, dto.password);
  }

  @Post('login')
  @HttpCode(200)                        // login is a POST but it does not create a resource
  login(@Body() dto: AuthDto) {
    return this.auth.login(dto.email, dto.password);
  }
}
```

## The auth module

auth.module.ts

```
@Module({
  imports: [
    UsersModule,
    JwtModule.registerAsync({
      global: true,                                  // JwtService available app-wide
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        secret: config.get<string>('JWT_SECRET'),
        signOptions: { expiresIn: '1h' },
      }),
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService],
})
export class AuthModule {}
```

## The guard

A guard implements `CanActivate` and returns `true` (continue) or throws/returns `false` (block). Ours verifies the bearer token and attaches the decoded payload to the request:

src/auth/auth.guard.ts

```
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly jwt: JwtService) {}

  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    const req = ctx.switchToHttp().getRequest();
    const [type, token] = req.headers.authorization?.split(' ') ?? [];
    if (type !== 'Bearer' || !token) throw new UnauthorizedException();
    try {
      req.user = await this.jwt.verifyAsync(token);       // { sub, email, role, iat, exp }
    } catch {
      throw new UnauthorizedException();
    }
    return true;
  }
}
```

## A custom parameter decorator

src/auth/current-user.decorator.ts

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export interface JwtUser { sub: number; email: string; role: string }

export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext): JwtUser => ctx.switchToHttp().getRequest().user,
);
```

## Protecting the Tasks controller

tasks.controller.ts (protected)

```
@UseGuards(AuthGuard)                // protects every route in this controller
@Controller('tasks')
export class TasksController {
  constructor(private readonly tasksService: TasksService) {}

  @Get()
  findAll(@CurrentUser() user: JwtUser, @Query() q: ListTasksQuery) {
    return this.tasksService.findAll(user.sub, q.page, q.limit);
  }

  @Post()
  create(@CurrentUser() user: JwtUser, @Body() dto: CreateTaskDto) {
    return this.tasksService.create(dto, user.sub);
  }

  @Patch(':id')
  update(@CurrentUser() user: JwtUser, @Param('id', ParseIntPipe) id: number,
         @Body() dto: UpdateTaskDto) {
    return this.tasksService.update(id, dto, user.sub);
  }
}
```

## Make everything protected by default

Forgetting `@UseGuards` on a new controller is a classic security bug. Flip the default: register the guard globally and mark public routes explicitly.

global guard with a @Public() opt-out

```
export const IS_PUBLIC = 'isPublic';
export const Public = () => SetMetadata(IS_PUBLIC, true);

@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly jwt: JwtService, private readonly reflector: Reflector) {}

  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC,
      [ctx.getHandler(), ctx.getClass()]);
    if (isPublic) return true;
    // ...verify the token exactly as before
    return true;
  }
}

// app.module.ts
providers: [{ provide: APP_GUARD, useClass: AuthGuard }]

// auth.controller.ts: mark the open endpoints
@Public() @Post('login') login(...) { ... }
```

## Role-based access control

The same pattern again: *a decorator stores metadata, a guard reads it*.

roles.guard.ts

```
export const ROLES_KEY = 'roles';
export const Roles = (...roles: string[]) => SetMetadata(ROLES_KEY, roles);

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(ctx: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride<string[]>(ROLES_KEY,
      [ctx.getHandler(), ctx.getClass()]);
    if (!required) return true;                          // no @Roles() means any logged-in user
    const { user } = ctx.switchToHttp().getRequest();    // set earlier by AuthGuard
    return required.includes(user.role);                 // false becomes 403 Forbidden
  }
}

// usage (AuthGuard must run first, so list it first)
@UseGuards(AuthGuard, RolesGuard)
@Roles('admin')
@Get('admin/tasks')
findEverything() { ... }
```

## Security checklist for auth

- Hash passwords with **bcrypt** or **argon2**. Never log or return them.
- Keep access tokens **short-lived** (minutes to an hour). Add refresh tokens for longer sessions.
- Never put secrets or sensitive personal data in a JWT: it is signed, not encrypted.
- Keep `JWT_SECRET` in the environment, long and random, different per environment.
- Always use HTTPS in production. Rate-limit `/auth/login` (see Episode 12).
- Return the same error for *unknown email* and *wrong password* so attackers cannot enumerate users.
- Need Google or GitHub login? `@nestjs/passport` offers ready-made strategies. The guard idea is identical.

EPISODE 10

# Interceptors, Filters and Middleware

*The cross-cutting toolbox that keeps controllers clean.*

## Interceptors

An **interceptor** wraps the handler. Code before `next.handle()` runs on the way in; operators piped onto the returned RxJS stream run on the way out. Perfect for timing, caching, logging and reshaping responses.

src/common/interceptors.ts

```
@Injectable()
export class TimingInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler): Observable<unknown> {
    const started = Date.now();
    return next.handle().pipe(
      tap(() => console.log(`${ctx.getHandler().name} took ${Date.now() - started}ms`)),
    );
  }
}

@Injectable()
export class WrapInterceptor implements NestInterceptor {
  intercept(_ctx: ExecutionContext, next: CallHandler) {
    return next.handle().pipe(map((data) => ({ data, ok: true })));   // { data, ok }
  }
}
```

## Exception filters

Nest already turns `HttpException` subclasses into JSON. A filter lets you control the shape (useful for consistent error contracts and for logging unexpected errors):

src/common/all-exceptions.filter.ts

```
@Catch()                                          // catch everything, not only HttpException
export class AllExceptionsFilter implements ExceptionFilter {
  private readonly logger = new Logger(AllExceptionsFilter.name);

  catch(exception: unknown, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    const req = host.switchToHttp().getRequest();
    const isHttp = exception instanceof HttpException;
    const status = isHttp ? exception.getStatus() : HttpStatus.INTERNAL_SERVER_ERROR;
    if (!isHttp) this.logger.error(exception);     // unexpected: log the stack

    res.status(status).json({
      success: false,
      status,
      message: isHttp ? exception.message : 'Internal server error',
      path: req.url,
      timestamp: new Date().toISOString(),
    });
  }
}
```

## Middleware

logger middleware

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, _res: Response, next: NextFunction) {
    console.log(`${req.method} ${req.originalUrl}`);
    next();                                          // do not forget this
  }
}

export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(LoggerMiddleware).forRoutes('*');
  }
}
```

## Where can each tool be attached?

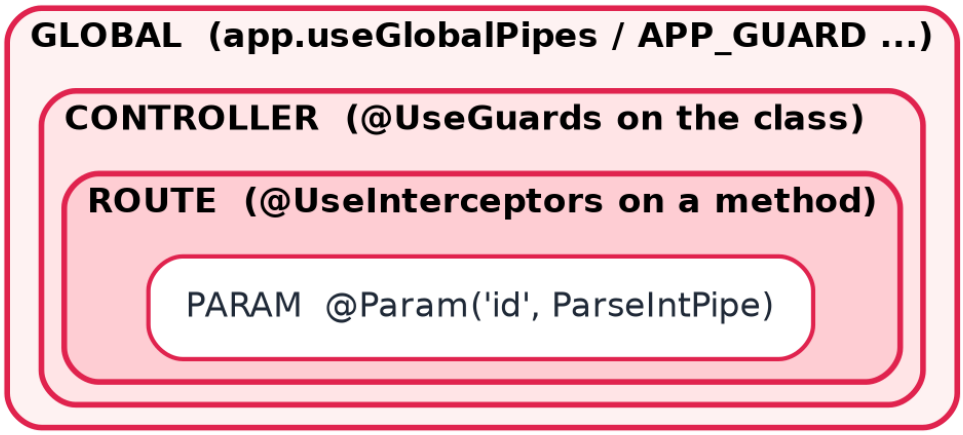



*Figure 8. Binding levels from widest to narrowest. Pick the narrowest level that does the job.*

| Level | How | Example |
|---|---|---|
| Global (no DI) | `app.useGlobalFilters(new X())` in `main.ts` | Simple filters and pipes |
| Global (with DI) | Provider with `APP_GUARD`, `APP_INTERCEPTOR`, `APP_PIPE` or `APP_FILTER` | Guards that need `JwtService` |
| Controller | `@UseGuards()` / `@UseInterceptors()` on the class | Protect one feature |
| Route | Same decorators on a method | Cache a single endpoint |
| Parameter | Pipe passed to `@Param()` / `@Query()` / `@Body()` | `ParseIntPipe` |

app.module.ts: global registration with dependency injection

```
providers: [
  { provide: APP_FILTER, useClass: AllExceptionsFilter },
  { provide: APP_INTERCEPTOR, useClass: TimingInterceptor },
]
```

> **WATCH OUT**
> `app.useGlobalGuards(new AuthGuard())` is created **outside** the DI container, so the guard cannot receive injected services. When a global tool needs dependencies, register it with the `APP_*` token inside a module instead.

EPISODE 11

# Testing Like a Professional

*Dependency injection makes testing cheap: swap real dependencies for fakes.*

Nest projects ship with **Jest** and **supertest**. There are two levels you will write:

| Type | What it tests | Speed | Dependencies |
|---|---|---|---|
| Unit test | One class in isolation (service, guard, pipe) | Milliseconds | Mocked |
| End-to-end (e2e) | The whole app over real HTTP | Seconds | Real module graph (test DB if needed) |

## Unit test: a service with a fake repository

src/tasks/tasks.service.spec.ts

```
describe('TasksService', () => {
  let service: TasksService;
  const repo = {
    findOne: jest.fn(),
    find: jest.fn(),
    save: jest.fn(),
    create: jest.fn((x) => x),
  };

  beforeEach(async () => {
    const moduleRef = await Test.createTestingModule({
      providers: [
        TasksService,
        { provide: getRepositoryToken(Task), useValue: repo },   // fake replaces the real repo
      ],
    }).compile();
    service = moduleRef.get(TasksService);
    jest.clearAllMocks();
  });

  it('throws NotFoundException when the task is missing', async () => {
    repo.findOne.mockResolvedValue(null);
    await expect(service.findOne(99, 1)).rejects.toThrow(NotFoundException);
  });

  it('only searches inside the owner\'s tasks', async () => {
    repo.findOne.mockResolvedValue({ id: 5, ownerId: 1 });
    await service.findOne(5, 1);
    expect(repo.findOne).toHaveBeenCalledWith({ where: { id: 5, ownerId: 1 } });
  });
});
```

## End-to-end test

test/tasks.e2e-spec.ts

```
import * as request from 'supertest';

describe('Tasks (e2e)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({ imports: [AppModule] }).compile();
    app = moduleRef.createNestApplication();
    // IMPORTANT: apply the same global setup as main.ts, or validation will not run
    app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
    await app.init();
  });

  afterAll(() => app.close());

  it('GET /tasks without a token returns 401', () =>
    request(app.getHttpServer()).get('/tasks').expect(401));

  it('POST /tasks with an empty title returns 400', async () => {
    const login = await request(app.getHttpServer())
      .post('/auth/login').send({ email: 'test@example.com', password: 'password123' });
    return request(app.getHttpServer())
      .post('/tasks')
      .set('Authorization', `Bearer ${login.body.access_token}`)
      .send({ title: '' })
      .expect(400);
  });
});
```

> **GOOD TO KNOW**
> Depending on your TypeScript setup you may need `import request from 'supertest'` instead of `import * as request`. If the e2e tests touch a database, point them at a **separate test database** and clean it between runs.

## Testing tips

- Test **behavior** (inputs and outputs), not private implementation details.
- Use `Test.createTestingModule` and `overrideProvider(X).useValue(fake)` to replace pieces of a real module.
- Test guards and pipes directly: construct them and call `canActivate` / `transform` with fake context.
- Run `npm run test:cov` occasionally to find untested branches, but do not chase 100 percent blindly.

EPISODE 12

# API Docs, Config and Production Polish

*The difference between a demo and something you can deploy.*

## Swagger / OpenAPI documentation

terminal

```
npm i @nestjs/swagger
```

main.ts

```
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';

const config = new DocumentBuilder()
  .setTitle('TaskFlow API')
  .setVersion('1.0')
  .addBearerAuth()                // adds the "Authorize" button for JWT
  .build();
SwaggerModule.setup('docs', app, SwaggerModule.createDocument(app, config));
```

Visit `/docs` for an interactive API explorer. Add `@ApiProperty()` to DTO fields and `@ApiBearerAuth()` / `@ApiTags('tasks')` to controllers for richer docs. The Swagger CLI plugin in `nest-cli.json` can infer most DTO metadata automatically.

## Validate environment variables at startup

A missing `JWT_SECRET` should crash the app on boot, not fail mysteriously on the first login:

app.module.ts (npm i joi)

```
import * as Joi from 'joi';

ConfigModule.forRoot({
  isGlobal: true,
  validationSchema: Joi.object({
    DATABASE_URL: Joi.string().required(),
    JWT_SECRET: Joi.string().min(16).required(),
    PORT: Joi.number().default(3000),
  }),
})
```

## Security and robustness

main.ts additions (npm i helmet)

```
import helmet from 'helmet';

app.use(helmet());                                          // secure HTTP headers
app.enableCors({ origin: ['https://app.example.com'] });    // lock down origins
app.enableShutdownHooks();                                  // graceful shutdown on SIGTERM
app.setGlobalPrefix('api');
```

rate limiting (option shape differs between versions; check the docs)

```
// npm i @nestjs/throttler
ThrottlerModule.forRoot([{ ttl: 60_000, limit: 60 }])      // 60 requests per minute per client
// then register ThrottlerGuard with APP_GUARD, and tighten limits on /auth/login with @Throttle()
```

## Logging, health and containers

- **Logging:** Nest has a built-in `Logger`. For structured JSON logs in production use `nestjs-pino`.
- **Health checks:** `@nestjs/terminus` exposes `/health` endpoints (database ping, memory) for Kubernetes and load balancers.

- **Migrations:** generate and run them with the TypeORM CLI instead of `synchronize`. Commit them to version control.

Dockerfile (multi-stage: small final image, no dev dependencies)

```
# use a current LTS image
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

## Pre-launch checklist

| Area | Check |
|---|---|
| Config | All secrets in env vars; startup validation; no `.env` committed |
| Database | `synchronize: false`; migrations run in CI/CD; connection pool sized |
| Security | `helmet`, CORS allow-list, rate limiting, HTTPS, short JWT expiry |
| Validation | Global `ValidationPipe` with `whitelist: true` |
| Errors | Global exception filter; unexpected errors logged with stack traces |
| Observability | Structured logs, health endpoint, metrics/tracing if needed |
| Operations | Graceful shutdown, Docker image, readiness and liveness probes |
| Quality | Unit and e2e tests in CI; lint and type-check on every pull request |

EPISODE 13

# Beyond REST: The Hero Tier

*The same modules, DI, guards and pipes power far more than HTTP.*

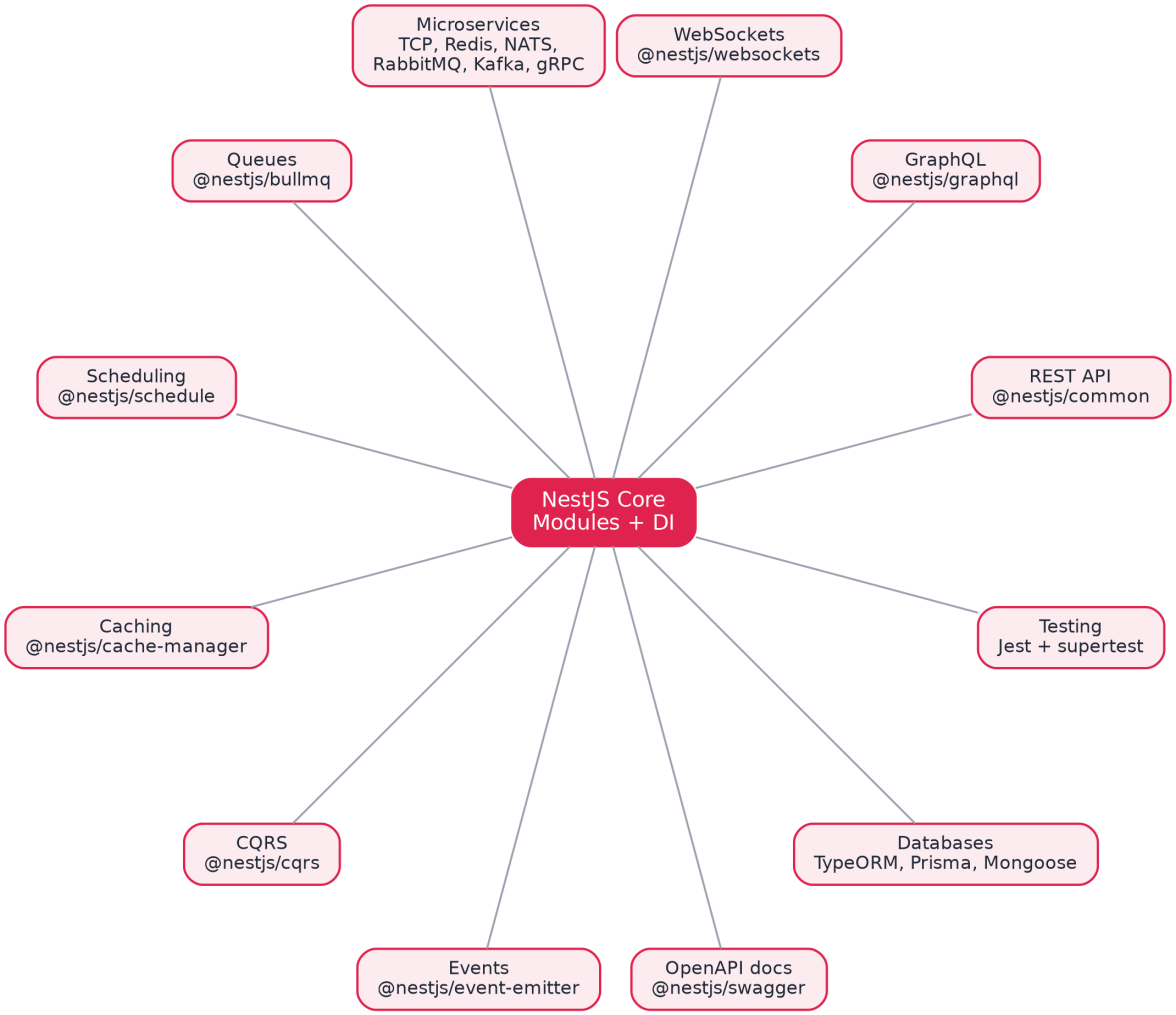



*Figure 9. The NestJS ecosystem. Everything plugs into the same module and dependency injection system you already know.*

| Feature | Package | Use it for |
|---|---|---|
| GraphQL | `@nestjs/graphql` | Code-first or schema-first APIs with resolvers |
| WebSockets | `@nestjs/websockets` | Real-time features (chat, live updates) via gateways |
| Microservices | `@nestjs/microservices` | TCP, Redis, NATS, RabbitMQ, Kafka, gRPC transports |
| Queues | `@nestjs/bullmq` | Background jobs (emails, reports, media processing) |
| Scheduling | `@nestjs/schedule` | Cron jobs and timers |
| Caching | `@nestjs/cache-manager` | `CacheInterceptor`, manual cache access |
| Events | `@nestjs/event-emitter` | Decoupled in-process events |
| CQRS | `@nestjs/cqrs` | Separate commands and queries for complex domains |

## WebSockets: live task updates

tasks.gateway.ts

```
// npm i @nestjs/websockets @nestjs/platform-socket.io
@WebSocketGateway({ cors: true })
export class TasksGateway {
  @WebSocketServer() server: Server;                // from 'socket.io'

  @SubscribeMessage('ping')
  handlePing(@MessageBody() data: string) {
    return { event: 'pong', data };                 // reply to the sender
  }

  broadcastCreated(task: Task) {
    this.server.emit('task:created', task);         // push to every connected client
  }
}
// Register TasksGateway in providers, inject it into TasksService, call broadcastCreated().
```

## Background jobs with a queue

queue processor and producer

```
// npm i @nestjs/bullmq bullmq   (needs Redis)
@Processor('emails')
export class EmailProcessor extends WorkerHost {
  async process(job: Job<{ to: string }>) {
    await this.mailer.send(job.data.to, 'Welcome to TaskFlow');   // slow work off the request path
  }
}

// producer side
constructor(@InjectQueue('emails') private readonly queue: Queue) {}
await this.queue.add('welcome', { to: user.email });
```

## Scheduling and events

cron job and event listener

```
@Injectable()
export class CleanupService {
  @Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT)      // npm i @nestjs/schedule
  async removeOldTasks() { /* ... */ }

  @OnEvent('task.created')                         // npm i @nestjs/event-emitter
  handleTaskCreated(task: Task) { /* send a notification */ }
}
// emit from a service: this.events.emit('task.created', task)
```

## Microservices

A microservice handler looks almost identical to a controller. Only the transport changes.

TCP microservice and client

```
// math-service/main.ts
const app = await NestFactory.createMicroservice<MicroserviceOptions>(AppModule, {
  transport: Transport.TCP,
  options: { port: 4000 },
});
await app.listen();

// math-service controller
@MessagePattern({ cmd: 'sum' })
sum(data: number[]) {
  return data.reduce((a, b) => a + b, 0);
}

// API gateway: call it through a client proxy
constructor(@Inject('MATH') private readonly client: ClientProxy) {}
sum(numbers: number[]) {
  return this.client.send({ cmd: 'sum' }, numbers);   // returns an Observable
}
```

## Monolith first, microservices later

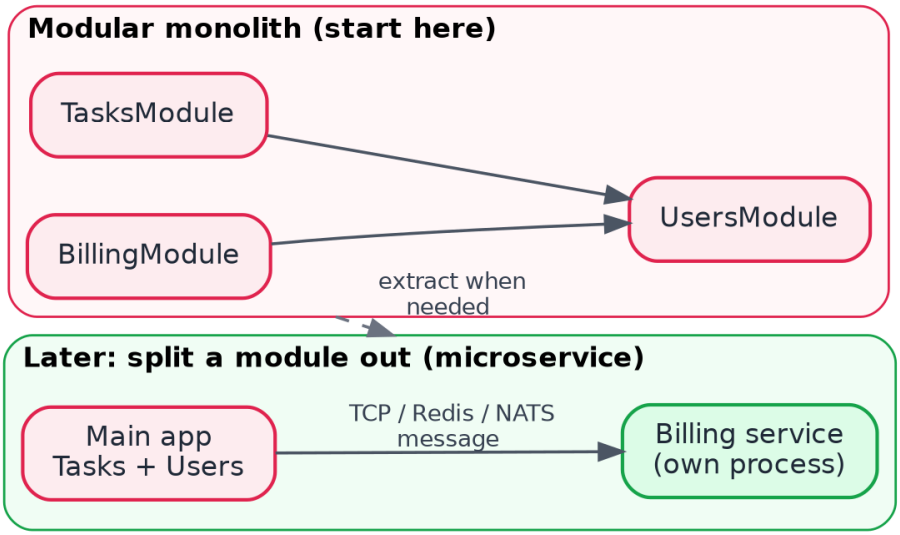



*Figure 10. Start as a modular monolith. If modules are cleanly separated, extracting one into its own service later is a contained change.*

**INSTRUCTOR TIP**

Do not begin with microservices because they sound modern. They add network failures, deployment complexity and distributed debugging. A well-structured modular monolith keeps your options open at a fraction of the cost.

## GraphQL in one glance

code-first GraphQL resolver (the @ObjectType()/@Field() decorators go on the entity)

```
@Resolver(() => Task)
export class TasksResolver {
  constructor(private readonly tasks: TasksService) {}

  @Query(() => [Task])
  tasks(@Args('ownerId', { type: () => Int }) ownerId: number) {
    return this.tasks.findAll(ownerId);
  }
}
```

EPISODE 14

# Architecture, Best Practices and Troubleshooting

*Habits that keep a Nest codebase healthy at 100,000 lines.*

## Organize by feature, not by type

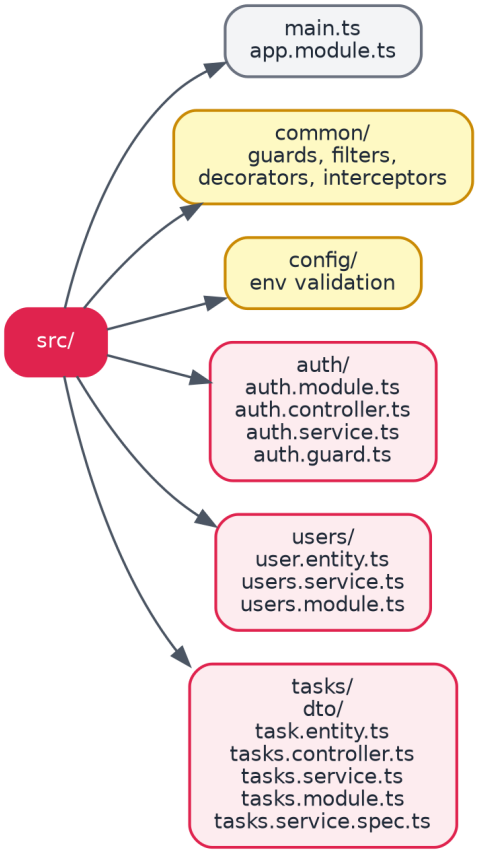



*Figure 11. Feature folders keep everything about tasks, users and auth together. Cross-cutting code lives in common/.*

## Rules I live by

1. **Thin controllers, fat services.** Controllers translate HTTP; services hold the logic and are easy to test.
2. **Never return entities that contain secrets.** Use response classes, `@Exclude()` or selected columns.
3. **One module per domain area** with a small public surface through `exports`.
4. **Avoid circular dependencies.** If A and B need each other, extract a third provider.
5. **Centralize configuration** in `ConfigService`. Do not sprinkle `process.env` through the code.
6. **Throw built-in HTTP exceptions** (`NotFoundException`, `ForbiddenException`...) from services; filters turn them into responses.
7. **Validate at the edge, trust inside.** DTOs plus `ValidationPipe` mean services receive clean data.
8. **Write the test when you write the service.** DI makes it cheap.
9. **Prefer constructor injection** and `private readonly`. Avoid property injection and service locators.

## Common errors and how to fix them

| Symptom | Likely cause | Fix |
|---|---|---|
| *Nest can't resolve dependencies of X (?)* | Provider missing from the module, or not exported/imported | Add it to `providers`, or `exports` it from its module and import that module |
| *Circular dependency* warning or crash | A imports B and B imports A | Extract shared logic; as a last resort use `forwardRef()` |
| Validation does nothing | No global `ValidationPipe`; DTO is an interface; no decorators | Register the pipe, use a class, add `class-validator` decorators |
| Extra fields reach the database | `whitelist` not enabled | `new ValidationPipe({ whitelist: true })` |
| `Cannot GET /route` (404) | Controller not in a module, or a `:id` route captures the path | Add to `controllers`; put static routes before dynamic ones |
| `@Param('id')` is a string | No conversion | `ParseIntPipe`, or `transform: true` with a typed DTO |
| *No metadata for entity* / table missing | Entity not registered | `forFeature([Entity])` plus `autoLoadEntities: true` |
| Everything returns 401 | Global guard without a `@Public()` opt-out for login | Mark login and register as public |
| Response is slow or memory grows | Request-scoped providers everywhere | Return to default scope; scope only what truly needs it |
| Guard cannot use `JwtService` | Global guard created with `new` in `main.ts` | Register it via `APP_GUARD` in a module |

## Performance notes

- Switch to the **Fastify adapter** (`@nestjs/platform-fastify`) when raw throughput matters.
- Avoid `Scope.REQUEST` unless you need it; it makes dependent providers request-scoped too.
- Cache expensive reads with `CacheInterceptor`; move slow work to a **queue**.
- Select only needed columns, add database **indexes** for fields you filter on, and paginate.
- Enable response **compression** (the `compression` middleware) and keep payloads small.

EPISODE 15

# Your Roadmap and Practice Challenges

*Knowledge sticks when you build. Here is a four-week plan.*

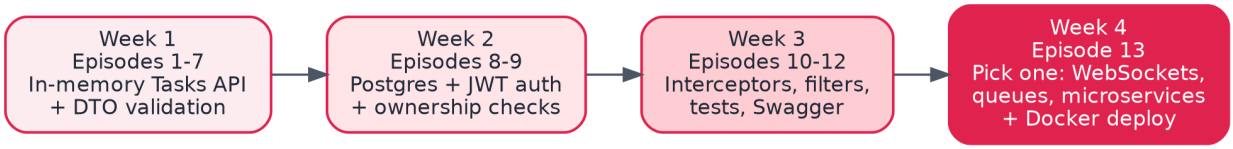



*Figure 12. A four-week path from first route to a deployed, tested, real-time API.*

## Practice challenges

1. Add `PATCH /tasks/:id/toggle` and a `?done=true` filter using a query DTO.
2. Add pagination metadata (`total`, `page`, `limit`, `hasNext`) and return it in a wrapper interceptor.
3. Build a `@Roles('admin')` decorator and an admin-only `GET /admin/tasks` listing every user's tasks.
4. Write an e2e test that registers, logs in, creates a task, and confirms another user cannot read it (expect 404 or 403).
5. Add refresh tokens with rotation and a logout endpoint that revokes them.
6. Add tags with a many-to-many relation and filter tasks by tag.
7. Broadcast `task:created` over a WebSocket gateway to the owner's connected clients only.
8. Send a welcome email through a BullMQ queue and make the job retry on failure.
9. Containerize the app with Docker Compose (API + PostgreSQL) and deploy it somewhere.

## Where to learn more

- **Official documentation**: docs.nestjs.com. It is excellent and has chapters for every topic here.
- **Source code**: github.com/nestjs/nest and the official `nestjs/nest` samples folder show working examples.
- **Read the framework's own modules.** `@nestjs/jwt` and `@nestjs/config` are small, readable examples of dynamic modules.

**INSTRUCTOR TIP**

You now have the full mental model: **modules** organize, **providers** do the work, **controllers** expose it, **DI** wires it together, and **pipes, guards, interceptors and filters** handle everything cross-cutting. Every other Nest feature is a variation on those ideas. Go build something.

APPENDIX A

# Cheat Sheets

*Keep this page open while you code.*

## Nest CLI

| Command | Generates |
|---|---|
| `nest new <name>` | A new project |
| `nest g resource <name>` | Module, controller, service, DTOs and entity (CRUD) |
| `nest g module <name>` / `controller` / `service` | Individual building blocks |
| `nest g guard <name>` | A guard (also: `pipe`, `interceptor`, `filter`, `middleware`, `decorator`) |
| `nest build` / `nest start --watch` | Compile / run with reload |

## Decorator quick reference

| Category | Decorators |
|---|---|
| Structure | `@Module` `@Controller` `@Injectable` `@Global` |
| Routing | `@Get` `@Post` `@Put` `@Patch` `@Delete` `@All` `@HttpCode` `@Header` `@Redirect` |
| Request data | `@Param` `@Query` `@Body` `@Headers` `@Req` `@Res` `@Ip` `@Session` |
| Cross-cutting | `@UseGuards` `@UseInterceptors` `@UsePipes` `@UseFilters` `@SetMetadata` |
| DI | `@Inject` `@Optional` `@InjectRepository` |
| Validation (class-validator) | `@IsString` `@IsInt` `@IsEmail` `@IsOptional` `@ValidateNested` `@MinLength` |
| Transformation (class-transformer) | `@Type` `@Exclude` `@Expose` `@Transform` |
| TypeORM | `@Entity` `@Column` `@PrimaryGeneratedColumn` `@OneToMany` `@ManyToOne` `@CreateDateColumn` |
| Swagger | `@ApiTags` `@ApiProperty` `@ApiBearerAuth` `@ApiResponse` |
| Microservices / WS | `@MessagePattern` `@EventPattern` `@WebSocketGateway` `@SubscribeMessage` |

## Built-in HTTP exceptions

| Class | Status |
| --- | --- |
| `BadRequestException` | 400 |
| `UnauthorizedException` | 401 |
| `ForbiddenException` | 403 |
| `NotFoundException` | 404 |
| `ConflictException` | 409 |
| `UnprocessableEntityException` | 422 |
| `InternalServerErrorException` | 500 |

## Built-in pipes

- `ValidationPipe`, `ParseIntPipe`, `ParseFloatPipe`, `ParseBoolPipe`, `ParseArrayPipe`, `ParseUUIDPipe`, `ParseEnumPipe`, `DefaultValuePipe`, `ParseFilePipe`

APPENDIX B

# Glossary

*Plain-English definitions of the vocabulary.*

| Term | Meaning |
|---|---|
| Decorator | A function attached with `@` to a class, method or parameter that adds metadata or behavior. |
| Module | A class marked `@Module()` that groups controllers and providers and declares imports and exports. |
| Controller | Class that maps routes to handler methods and shapes the HTTP response. |
| Provider | Any class (or value) the container can inject: services, repositories, factories. |
| Dependency Injection | A class declares what it needs; an external container supplies it. |
| IoC container | The Nest runtime component that builds and stores provider instances. |
| Singleton | A single shared instance (the default provider scope). |
| DTO | Data Transfer Object: a class describing data crossing a boundary, often validated. |
| Pipe | Validates or transforms input before the handler runs. |
| Guard | Decides if a request may proceed (authentication, authorization). |
| Interceptor | Wraps handler execution to add behavior before and after (RxJS based). |
| Exception filter | Catches thrown errors and builds the error response. |
| Middleware | Function that runs before routing, with access to the raw request and response. |
| Entity | A class mapped to a database table (TypeORM). |
| Repository | Object offering read and write operations for one entity. |
| JWT | JSON Web Token: a signed token carrying claims such as the user id. |
| Dynamic module | A module configured at import time via `forRoot()`, `register()` or `forFeature()`. |
| Transport | The channel a microservice uses to talk (TCP, Redis, NATS, Kafka...). |